

CMOS Matrix Multiplication Mini-Project

Design A vs Design B vs Design C — architecture, cycles, requests, stalls and synthesis results

1. Final Synthesis Comparison

Metric	Design A Parallel	Design B K-serial	Design C K-serial + double buffer
Cells	113,160	62,151	113,160
Logic depth	71	57	71
Area × delay	8,034,360	3,542,607	8,034,360
W1 cycles	40,961	53,249	41,729
W1 stalls	1	12,289	769
W1 errors	0	0	0
W2 cycles	299,009	397,313	299,777
W2 stalls	1	98,305	769
W2 errors	0	0	0

Important: All three designs passed W1 and W2 with zero errors. Design C's reported synthesis footprint is the same as Design A in the current implementation, while its double buffering gives near-reference throughput.

2. Common Specification

Operation: $C[4][4] += A[4][4] \times B[4][4]$. A and B are signed 8-bit values; C is a signed 32-bit accumulator.

Per 4×4 tile: 64 MAC operations = $4 \times 4 \times 4$. The interface needs 8 load requests (4 A rows + 4 B rows) plus 1 Mul request = **9 requests/tile**.

Workload	Output tiles	K-tiles / output tile	Mul operations	Requests
W1	256	16	4,096	40,960
W2	256	128	32,768	299,008

3. Design A — Fully Parallel Reference

Design A calculates all four K products for every $C[i][j]$ in one Mul cycle. There are 16 output elements and 4 products per output, giving **64 multipliers** operating in parallel. The four products for each output are combined by an adder tree.

Mul latency: 1 computation cycle.

Hardware: 64 multipliers + 16 four-input accumulation trees.

W1: 40,960 requests, 40,961 cycles, 1 stall, 0 errors.

W2: 299,008 requests, 299,009 cycles, 1 stall, 0 errors.

Synthesis: 113,160 cells; logic depth 71; area×delay 8,034,360.

Interpretation: Design A gives the lowest computation latency but uses the largest amount of parallel arithmetic hardware.

4. Design B — K-Serial

Design B keeps all 16 output elements parallel but serializes the K dimension. Only one K slice is computed per cycle, so only **16 multipliers** are needed.

Mul sequence: cycle 1 $\rightarrow K=0$; cycle 2 $\rightarrow K=1$; cycle 3 $\rightarrow K=2$; cycle 4 $\rightarrow K=3$. Thus one Mul takes 4 computation cycles.

Because the DUT is busy for those four cycles, the next A/B tile cannot be loaded during the computation. Relative to Design A, every Mul adds 3 extra cycles. For W1: $3 \times 4,096 = 12,288$ extra cycles; measured difference = $53,249 - 40,961 = 12,288$. For W2: $3 \times 32,768 = 98,304$ extra cycles; measured difference = $397,313 - 299,009 = 98,304$.

Synthesis: 62,151 cells; logic depth 57; area×delay 3,542,607.

W1: 53,249 cycles, 12,289 stalls, 0 errors.

W2: 397,313 cycles, 98,305 stalls, 0 errors.

Interpretation: Design B greatly reduces arithmetic hardware and logic depth, but pays for it with lower throughput.

5. Design C — K-Serial with Double Buffering

Design C keeps the 16-multiplier K-serial engine from Design B but adds two A/B storage banks: **Bank 0 = A0/B0** and **Bank 1 = A1/B1**.

The active bank is used for multiplication while the inactive bank receives the next tile. After K=3 completes, the active-bank bit flips. The next tile can therefore start computation without waiting for all of its loads to occur after the previous computation.

Compute sequence: K=0, K=1, K=2, K=3 → 4 computation cycles per Mul.

Overlap: current bank computes while the other bank loads A/B.

W1: 40,960 requests → 41,729 cycles → 769 stalls → 0 errors.

W2: 299,008 requests → 299,777 cycles → 769 stalls → 0 errors.

The key result is W2: Design C needs only 299,777 cycles versus 397,313 for Design B, while Design A needs 299,009. Thus Design C is only **768 cycles above the reference** for W2. This shows that most of the four-cycle K-serial computation is hidden behind loading the next tile.

Synthesis: 113,160 cells; logic depth 71; area×delay 8,034,360 in the current implementation.

6. Why the Stall Counts Differ

Design A: Mul completes in one cycle, so only 1 measured stall occurs in each workload.

Design B: each Mul occupies four cycles and blocks the next tile's loads. The three additional computation cycles per Mul accumulate: 12,288 extra W1 cycles and 98,304 extra W2 cycles.

Design C: the inactive bank can load while the active bank computes. Therefore the K-serial latency is largely hidden. Both workloads show only 769 stalls, dominated by startup/drain and boundary effects.

7. Direct Design Trade-off

Feature	Design A	Design B	Design C
Multipliers	64	16	16
A/B banks	1	1	2
Mul computation	1 cycle	4 cycles	4 cycles
Load during Mul?	Not needed	No	Yes, inactive bank
W1 cycles	40,961	53,249	41,729
W2 cycles	299,009	397,313	299,777
Cells	113,160	62,151	113,160
Logic depth	71	57	71

8. Final Conclusion

Design A is the maximum-parallelism reference: fastest computation but highest arithmetic cost. Design B is the area-oriented design: it reduces the multiplier count from 64 to 16 and reduces the synthesized cell count and logic depth, but its four-cycle Mul operations cannot overlap with loading. Design C adds double buffering to recover the lost throughput. It retains the 16-multiplier K-serial engine while allowing the next A/B tile to load during computation. The measured W2 result is especially strong: 299,777 cycles versus 299,009 for Design A and 397,313 for Design B. All tested workloads passed with zero errors.

Viva one-line summary: A uses parallelism to minimize latency; B trades latency for area by serializing K; C uses double buffering to hide B's serialization latency by overlapping computation with loading.